

Riftbound Overlay System

A real-time streaming overlay system for OBS, controllable from any browser anywhere in the world.

How It Works

```
[You, anywhere in the world]
  ↓ (browser)
[controller.html] ↔ [Render Server] ← [OBS Browser Sources]
                        ↓
                    score-overlay.html
                    player1-decklist.html
                    player2-decklist.html
                    starting-soon.html
```

The Node.js server on Render is the relay. The controller sends events to it, and the overlays running inside OBS receive them in real time. Card data is bundled as a static JSON file — no external API calls at runtime.

Free tier note: Render spins the server down after 15 minutes of no traffic. The first request after that takes ~30 seconds to wake up. Open your controller URL a minute or two before going live to wake it up.

One-Time Setup

Step 1 — Clone or download the repo

```
git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git
cd YOUR_REPO
npm install
```

Step 2 — Fetch the card data

Card images and data come from the Riftcodex API. Run this script once locally to download all card data into a static file:

```
node fetch-cards.js
```

This creates `public/cards.json` (~2–5 MB). Commit it:

```
git add public/cards.json
git commit -m "Add card data"
git push
```

When a new set is released: run `node fetch-cards.js` again, then commit and push the updated `public/cards.json`.

Step 3 — Create a GitHub repo (if starting from scratch)

1. Go to <https://github.com/new> and create a new repository
2. Push the project up:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main
```

Step 4 — Deploy on Render

1. Create a free account at <https://render.com>
2. Click **New** → **Web Service**
3. Connect your GitHub repository
4. Fill in the settings:

- **Runtime:** Node
 - **Build Command:** `npm install`
 - **Start Command:** `node server.js`
 - **Instance Type:** Free
5. Click **Create Web Service**
 6. Render builds and deploys — takes ~2 minutes the first time
 7. Your URL appears at the top: `https://your-app.onrender.com`

Step 5 — Set your room password

In the Render dashboard → your service → **Environment** tab → **Add Environment Variable**:

```
ROOM_PASSWORD = your_secret_password_here
```

Default if not set: `riftbound2025`

Click **Save Changes** — Render redeploys automatically.

Step 6 — Add OBS Browser Sources

In OBS, add a **Browser Source** for each overlay. Set all to **1920×1080** with a transparent/black background.

Scene	Source Name	URL
Starting Soon	Starting Soon	<code>https://your-app.onrender.com/starting-soon.html</code>
Match	Score Overlay	<code>https://your-app.onrender.com/score-overlay.html</code>
Match	P1 Decklist	<code>https://your-app.onrender.com/player1-decklist.html</code>
Match	P2 Decklist	<code>https://your-app.onrender.com/player2-decklist.html</code>

OBS Browser Source settings to check:

- Width: 1920, Height: 1080
- ☒ Shutdown source when not visible
- ☒ Refresh browser when scene becomes active

These only need to be set up once — they always load the latest version on Render automatically.

Using the Controller

Open `https://your-app.onrender.com/controller.html` in any browser. Enter your room password to authenticate.

Image Gallery tab

Browse all Riftbound cards organized by type: **Legends, Battlefields, All Cards**.

- Click any card to send it to the **Spotlight** panel (bottom-right of the score overlay)
- Use the **P1 TARGET** / **P2 TARGET** buttons to select which player a legend or battlefield applies to
- Use the search bar to filter by card name
- Cards are paginated 50 per page

Scoreboard tab

Control	What it does
Player name fields	Updates player names live on the score overlay
+ / - buttons	Increments or decrements game wins
Round field	Updates the round label (e.g. "Round 1 of 5")
Timer	Set and start/stop the match countdown
Legend selector	Sends the selected legend portrait to the score overlay
Battlefield selector	Sends the selected battlefield to the score overlay

Decklist tab

Paste a decklist in the text area for Player 1 or Player 2 and click **Send**. The decklist overlay for that player will populate with card images automatically.

Decklist format:

```
Legend:
1 Ahri - Nine-Tailed Fox

Champion:
1 Ahri

MainDeck:
3 Acceptable Losses
2 Blade Whirl
...

Battlefields:
1 Abandoned Hall

Runes:
1 Attunement
```

Updating Card Data (New Sets)

When Riftbound releases a new set, update the card database:

```
node fetch-cards.js
git add public/cards.json
git commit -m "Update cards - new set"
git push
```

Render redeploys automatically. Done.

Local Development

```
npm install
node server.js
```

Open <http://localhost:3000/controller.html> — everything works the same as production.

Project Structure

```
poro-overlay/
├─ server.js           # Express + Socket.IO relay server
├─ fetch-cards.js      # Run locally to update card data
├─ package.json
└─ public/
    ├─ cards.json       # Static card database (committed to repo)
    ├─ cardDatabase.js  # Shared card lookup module
    ├─ controller.html  # The control panel
    ├─ score-overlay.html # Score + legend + battlefield overlay
    ├─ player1-decklist.html # P1 full decklist overlay
    ├─ player2-decklist.html # P2 full decklist overlay
    ├─ starting-soon.html # Stream starting soon screen
    ├─ decklist-overlay.html # Combined decklist overlay (alternative)
    ├─ images/
    │   └─ placeholder.png
    └─ fonts/
        └─ Cinzel-VariableFont_wght.ttf
```